# Analysis of Pareto-Inspired GNN-Based Ranking

## Contents


**1 Overview** **3**
- 1.1 Line references . . . 3

**2 Mathematical formulation** **3**

**3 Feature preprocessing and normalization** **4**
- 3.1 Log1p transform . . . 4
- 3.2 Robust scaling . . . 4
- 3.3 Standard scaling . . . 4
- 3.4 Min–Max proxy for compass and ranking supervision . . . 5

**4 Graph construction and manifold structure** **5**
- 4.1 The $k$NN graph . . . 5
- 4.2 Positional encoding . . . 5

**5 Graph neural network architecture** **6**
- 5.1 Message passing and aggregation . . . 6
- 5.2 Residual GCN layer . . . 6
- 5.3 Activation choices . . . 7
- 5.4 ManifoldRanker as a multi-task model . . . 7

**6 Training objectives and loss terms** **7**
- 6.1 Compass loss . . . 7
- 6.2 Reconstruction loss . . . 8
- 6.3 Pairwise ranking loss . . . 8
- 6.4 Geometric smoothness loss . . . 8
- 6.5 Total loss and weighting . . . 9

**7 VRAM-awareness and optimization strategy** **10**

**8 Code walkthrough** **10**
- 8.1 Imports and global utilities . . . 10
- 8.2 Design notes and stated limitations . . . 11
- 8.3 Configuration class `Config` . . . 11
  - 8.3.1 Hardware selection and reproducibility . . . 11
  - 8.3.2 Dataset configuration . . . 11
  - 8.3.3 Neural architecture dimensions . . . 12
  - 8.3.4 Graph topology and VRAM constraints . . . 12

8.3.5 Optimization hyperparameters . . . 12
8.3.6 Loss weights . . . 12
8.4 Data engine `MunicipalityManifold` . . . 12
8.5 Graph topology `SafeGraphEngine` . . . 13
8.6 Graph layer `ResGCNLayer` . . . 14
8.7 Model `ManifoldRanker` . . . 16
8.8 Execution `main` . . . 16

**9 Critical observations and potential improvements** **17**

**A Source Code** **18**

# 1 Overview

The notebook implements a VRAM-safe graph neural ranking pipeline for Brazilian municipalities. Each municipality is treated as a node with a fixed feature vector of telecommunications and infrastructure indicators. A $k$-nearest-neighbor graph is built in feature space, and a graph neural network (GNN) assigns each municipality a scalar utility score in $[0, 1]$. Sorting these scores produces a global ranking.

The code is organized into (1) configuration (`Config`) -hyperparameters, device selection, loss weights, and reproducibility settings-, (2) data engine (`MunicipalityManifold`) -CSV loading, numeric casting, and normalization pipelines-, (3) graph topology construction (`SafeGraphEngine`) -CPU-side $k$NN indexing, positional encodings, and memory-safe subgraph sampling-, (4) model (`ResGCNLayer`, `ManifoldRanker`) -residual graph convolution layers and two heads (ranking and reconstruction)-, and (5) execution (`main`) -training loop with streaming subgraphs to GPU, then inference in micro-batches.

## 1.1 Line references

We refer to line numbers in the Python script obtained by concatenating the notebook cells in order. The full source code with line numbers is included in Appendix A.

| **Module / section** | **Line range** |
|---|---|
| Imports | 1–18 |
| Design notes and TODOs | 21–33 |
| Global configuration | 35–159 |
| Data engine: MunicipalityManifold | 164–286 |
| Graph topology: SafeGraphEngine | 291–462 |
| Graph layer: ResGCNLayer | 468–544 |
| Model: ManifoldRanker | 546–668 |
| Execution: main | 674–895 |

# 2 Mathematical formulation

Let $N$ be the number of municipalities (rows in the dataframe after preprocessing) and $D$ the number of numeric indicators used as features. The code selects $D = 19$ feature columns (see `MunicipalityManifold.__init__`, lines 176–186). For each municipality $i \in \{1, \ldots, N\}$, we denote the raw feature vector by $x_i \in \mathbb{R}^D$. The full dataset is the matrix $X \in \mathbb{R}^{N \times D}$ whose $i$-th row is $x_i^\top$.

The pipeline constructs two normalized versions of these features:

- $X^{\text{model}} \in \mathbb{R}^{N \times D}$: standardized features for the neural network input and for neighbor search in the $k$NN graph.
- $X^{\text{pareto}} \in \mathbb{R}^{N \times D}$: Min–Max scaled features in $[0, 1]$ used only to form a weak "quality proxy" that orients the ranking losses.

The model outputs a scalar score $s_i \in [0, 1]$ for each municipality $i$. A global ranking is obtained by sorting $s_i$ in descending order. Because the ranking is derived from a single real-valued score, the induced order is transitive by construction: if $s_A > s_B$ and $s_B > s_C$, then necessarily $s_A > s_C$. This is the meaning of the comment in `ManifoldRanker.forward` (lines 613–616).

The notebook's filename and comments refer to "Pareto". In a multi-criteria setting, a common partial order is Pareto dominance. For two municipalities $A$ and $B$ with feature vectors $x_A, x_B \in \mathbb{R}^D$ (assuming "higher is better" for every feature), $A$ dominates $B$ if $x_{A,j} \geq x_{B,j}$ for all $j$ and $x_{A,j} > x_{B,j}$ for at least one $j$. The code does not compute Pareto dominance explicitly, because many municipality pairs are incomparable by standard Pareto partial ordering; instead it builds a scalar proxy $u_i = \frac{1}{D}\sum_{j=1}^{D} X_{i,j}^{\text{pareto}}$ (see training loop, line 769). This is a simple linear scalarization (equal weights) that converts a multi-criteria problem into a total order. Such scalarizations are easy to optimize but can differ from the true Pareto structure: two non-dominated points can be ordered arbitrarily by the mean.

# 3 Feature preprocessing and normalization

The preprocessing pipeline is implemented in `MunicipalityManifold._process_features` (lines 250–286). It aims to transform heavy-tailed, heterogeneous infrastructure indicators into a representation suitable for neural optimization.

## 3.1 Log1p transform

The first step is

$$X^{\log} = \log(1 + X^{\text{raw}}),$$

implemented as `np.log1p(X_raw)` (line 264). The transform has two purposes:

- It reduces the dynamic range of features that follow power-law or log-normal-like distributions (which are common among infrastructure data).
- The `+1` ensures numerical safety for zeros: $\log(0)$ is undefined but $\log(1) = 0$.

Mathematically, for large values $x$, $\log(1 + x)$ grows sublinearly, which prevents a few extreme municipalities from dominating gradients.

## 3.2 Robust scaling

After log compression, the code applies `RobustScaler` with `quantile_range=(5,95)` (lines 270–271). Robust scaling standardizes each feature using robust statistics:

$$\tilde{x}_{i,j} = \frac{x_{i,j}^{\log} - \text{median}(x_{:,j}^{\log})}{Q_{0.95}(x_{:,j}^{\log}) - Q_{0.05}(x_{:,j}^{\log})},$$

where $Q_q(\cdot)$ denotes the $q$-quantile. The $Q_q$-function is the inverse function to a CDF, finding the support value of the CDF that leads to a specific probability $q$. Using the 5th and 95th percentiles instead of the default interquartile range (25th–75th) makes the scaling less sensitive to extreme outliers while still using most of the distribution's mass.

## 3.3 Standard scaling

Finally, `StandardScaler` is applied to the robust-scaled features (lines 277–278):

$$X_{i,j}^{\text{model}} = \frac{\tilde{x}_{i,j} - \mu_j}{\sigma_j}, \qquad \mu_j = \frac{1}{N}\sum_{i=1}^{N} \tilde{x}_{i,j}, \quad \sigma_j^2 = \frac{1}{N}\sum_{i=1}^{N} (\tilde{x}_{i,j} - \mu_j)^2.$$

Centering to zero mean and unit variance improves conditioning for first-order optimizers (AdamW in our case) and makes learning rates more interpretable.

### 3.4 Min–Max proxy for compass and ranking supervision

In parallel, the code constructs a proxy matrix for "compass" guidance (compass here refers to the general direction of ranking) by applying Min–Max scaling to $X^{\log}$ (lines 283–284):

$$X_{i,j}^{\text{pareto}} = \frac{x_{i,j}^{\log} - \min_i x_{i,j}^{\log}}{\max_i x_{i,j}^{\log} - \min_i x_{i,j}^{\log}} \in [0,1].$$

Then a scalar proxy utility is defined as the mean across features $u_i = \frac{1}{D}\sum_j X_{i,j}^{\text{pareto}}$. This proxy is not a true Pareto ranking; it is a heuristic that provides a consistent direction for training in the absence of labeled rankings.

## 4 Graph construction and manifold structure

The GNN requires a graph $G = (V, E)$ over the municipalities. The notebook constructs $G$ as a $k$-nearest-neighbor graph in the normalized feature space $X^{\text{model}}$ using `sklearn.neighbors.NearestNeighbors`. This is implemented in `SafeGraphEngine.__init__` (lines 310–372).

### 4.1 The $k$NN graph

Let $x_i^{\text{model}}$ be row $i$ of $X^{\text{model}}$. For each node $i$, the code computes the indices of its $k$ nearest neighbors with Euclidean distance. This yields an index matrix $\mathcal{I} \in \{1, \ldots, N\}^{N \times k}$ such that $\mathcal{I}_{i,:}$ is the neighbor list for node $i$ (line 337).

In `sklearn`, nearest-neighbor relationships are directional: "$j$ is among the $k$ nearest neighbors of $i$" does not imply the converse. The resulting graph is therefore directed unless symmetrized explicitly. The code keeps it directed: edges are generated as $(i \to j)$ for each $j \in \mathcal{I}_{i,:}$ (lines 438–440). Degree normalization in `ResGCNLayer` (lines 523–532) uses the in-degree induced by these directed edges.

In many $k$NN implementations, each point is its own nearest neighbor at distance 0. If this occurs here, then the edge list includes self-loops $(i \to i)$. Self-loops interact with the residual path in `ResGCNLayer`: the node's own features may be added twice (once via aggregation through a self-loop and once through the explicit skip connection). Whether this is desirable is still uncertain; however, as sof now, the code does not remove self-loops.

### 4.2 Positional encoding

Message-passing GNNs are permutation-invariant with respect to node ordering: relabeling nodes does not change the computation as long as edges and features are relabeled consistently. While this invariance is often desirable, it also means the GNN has no notion of "global coordinates" on the dataset manifold. The notebook injects global structure through a positional encoding $p_i \in \mathbb{R}^{d_{\text{pe}}}$ computed from $X^{\text{model}}$ and concatenated to the node features at input (lines 562–566 and 623–626).

If the dataset size is $N \leq 15000$ (this will always be the case, as the number of municipalities is 5570), the code tries `SpectralEmbedding` (lines 356–364), which corresponds to Laplacian eigenmaps. At a high level, this means building a graph (here also based on nearest neighbors), forming a graph Laplacian $L$ (or a normalized variant), and using the bottom non-trivial eigenvectors of $L$ as coordinates in $\mathbb{R}^{d_{\text{pe}}}$. This produces an embedding that preserves local neighborhoods and often captures non-linear manifold structure more effectively than linear methods.

If spectral embedding is skipped or fails, the code falls back to PCA (lines 366–369). PCA finds orthogonal directions of maximum variance in $X^{\text{model}}$ and projects onto the top components. This

is a linear approximation to manifold coordinates. Additionally, if $N > 15000$ the code intentionally forces the PCA path (line 357) to avoid the heavier spectral method.

Regardless of how $p_i$ is computed, it is standardized with `StandardScaler` (line 372). This is important because the model concatenates $x_i$ and $p_i$: if $p_i$ had much larger magnitude, the input projection would prioritize positional information over actual domain features, which is typically undesirable.

# 5 Graph neural network architecture

The model is defined by two classes: `ResGCNLayer` (lines 468–544) and `ManifoldRanker` (lines 546–668). This section explains the exact computations performed and their theoretical motivation.

## 5.1 Message passing and aggregation

A common abstraction for GNNs is message passing. Given node representations $h_i^{(\ell)}$ at layer $\ell$, a generic message-passing update has the form

$$m_i^{(\ell)} = \mathrm{AGG}\big(\{\, \phi(h_j^{(\ell)}) : (j \to i) \in E \,\}\big), \qquad h_i^{(\ell+1)} = \psi\Big(h_i^{(\ell)}, m_i^{(\ell)}\Big),$$

where $\phi$ is a learnable transformation (often linear), AGG is a permutation-invariant aggregator such as sum or mean, and $\psi$ combines the aggregated neighborhood information with the node's current state.

In this code, $\phi$ is the linear map `self.proj` (line 483), AGG is mean pooling implemented via scatter-add followed by degree normalization (lines 520–532), and $\psi$ is a residual addition followed by GELU (line 544).

## 5.2 Residual GCN layer

`ResGCNLayer.forward` implements the following update. Let $x \in \mathbb{R}^{n \times d_{\text{in}}}$ be the node feature matrix for a sampled subgraph with $n$ nodes, and let `edge_index` be a COO-format edge list (a coordinate-list format) with two integer arrays $(\text{row}, \text{col})$ of length $|E|$ (line 501). The layer computes:

$$\begin{aligned}
H &= XW_{\text{proj}}^{\top} + b_{\text{proj}} \qquad \text{(linear projection)} \\
\text{out}_i &= \sum_{(j \to i) \in E} H_j \qquad \text{(scatter sum into destinations)} \\
\deg_i &= |\{\, (j \to i) \in E \,\}| \qquad \text{(in-degree)} \\
\bar{H}_i &= \frac{\text{out}_i}{\deg_i + \varepsilon} \qquad \text{(mean aggregation)} \\
X' &= \mathrm{GELU}\big(\bar{H} + \mathrm{skip}(X)\big)\,.
\end{aligned}$$

The skip path is either the identity or a linear projection depending on whether dimensions match (line 487). This residual structure is motivated by residual connections providing short gradient paths, which improve trainability of deeper networks, and mitigating oversmoothing, meaning that repeated neighborhood averaging can cause node representations to converge. Residual terms preserve node-specific information.

### 5.3 Activation choices

The code uses three different activations:

- GELU in the graph layers and input projection (lines 625 and 544). GELU is smooth and non-linear, and is common in transformer-style architectures.
- ELU in the decoder (line 587). ELU provides non-linearity while avoiding zero gradients for negative inputs (unlike ReLU).
- Tanh inside the ranking MLP (line 605). Tanh bounds latent coordinates to $[-1, 1]$, which can help interpret them as manifold coordinates, but it can saturate for large magnitudes and cause gradient shrinkage.

There are ATTENTION comments about these activations (lines 538–542, 580–584, 596–602) in the code, because activation choice affects monotonicity, saturation, and the smoothness of the learned scoring function. From a strictly mathematical standpoint, GELU, ELU, and Tanh are all monotone increasing, but they differ in curvature and asymptotics.

### 5.4 ManifoldRanker as a multi-task model

`ManifoldRanker` combines graph encoding with two output heads: (1) the ranking head, which maps the final node embedding to a scalar score $s_i \in [0, 1]$ via an MLP and sigmoid (line 635), and (2) the reconstruction head, which maps the final node embedding back to the original feature space (line 636). Multi-task training with reconstruction regularizes the embedding: it discourages degenerate solutions where scores satisfy ranking constraints but the intermediate representation discards most input information.

## 6 Training objectives and loss terms

The training loop constructs four losses and combines them as a weighted sum (lines 771–806). This section defines each term and explains its role.

### 6.1 Compass loss

For a batch of center nodes $\mathcal{B}$ (size `BATCH_SIZE`), the model produces scores $s_i$ for $i \in \mathcal{B}$. The code defines a weak proxy target

$$u_i = \frac{1}{D} \sum_{j=1}^{D} X_{i,j}^{\text{pareto}},$$

and penalizes deviations via mean squared error (line 773):

$$L_{\text{compass}} = \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} (s_i - u_i)^2.$$

The interpretation for this term is that it points the model toward a direction consistent with "more is better" across features, but we must bear in mind that it is only as reliable as the proxy $u_i$.

### 6.2 Reconstruction loss

The decoder predicts reconstructed features $\hat{x}_i \in \mathbb{R}^D$ for nodes $i$. The code measures MSE between reconstructed and input standardized features for center nodes only (line 777):

$$L_{\text{recon}} = \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \|\hat{x}_i - x_i^{\text{model}}\|_2^2.$$

Because $x^{\text{model}}$ is standardized, the scale of this loss is comparable across features. This is a form of autoencoder regularization.

### 6.3 Pairwise ranking loss

A margin-based pairwise ranking loss (`nn.MarginRankingLoss`, lines 726–732 and 779–792) is used. This means that, for two items $A$ and $B$ with scores $s_A, s_B$ and a desired ordering label $y \in \{-1, +1\}$, the loss is given by

$$\ell_{\text{margin}}(s_A, s_B, y) = \max(0, m - y(s_A - s_B)),$$

where $m > 0$ is a margin hyperparameter (`Config.MARGIN`). If $y = +1$, the loss enforces $s_A \geq s_B + m$. If $y = -1$, it enforces $s_B \geq s_A + m$.

In the code, random pairs are constructed within each batch by a random permutation (lines 782–785). Labels are derived from the proxy utilities $u_A, u_B$ (line 787):

$$y = \text{sign}(u_A - u_B).$$

The point of the margin, thus, is to take into account the fact that pairs with small proxy differences should be masked out as "too noisy" (lines 789–791).

**Possible implementation problem:** `margin_loss_fn` is instantiated with default reduction `'mean'`. In PyTorch this means `margin_loss_fn(s_A,s_B,target)` returns a single scalar already averaged over pairs. Multiplying this scalar by the vector mask (line 792) does not remove masked pairs from the reduction; it only rescales the final scalar by the mean mask value. If true masking were to be performed, the loss should be created with `reduction='none'` and reduced manually. This needs to be carefully checked in the current implementation.

### 6.4 Geometric smoothness loss

The geometric loss encourages nearby municipalities in the graph to have similar scores (lines 794–800):

$$L_{\text{geo}} = \frac{1}{|E_{\text{sub}}|} \sum_{(i,j) \in E_{\text{sub}}} (s_i - s_j)^2.$$

This is the Dirichlet energy of the score function on the graph and is closely related to graph Laplacian regularization: for an undirected graph, $L_{\text{geo}}$ is proportional to $s^\top L s$ where $L$ is the graph Laplacian. In directed graphs, the same expression still encourages consistency along directed edges.

In particular, the Dirichlet energy is a measure of the variability of a function. In the context of the `ManifoldRanker` model, it serves as the mathematical basis for the geometric loss ($L_{\text{geo}}$), enforcing smoothness on the graph structure. In classical mathematical physics, for a scalar field

$f : \Omega \to \mathbb{R}$ defined on a region $\Omega$, the Dirichlet energy $E[f]$ is defined as the integral of the squared magnitude of the gradient:

$$E[f] = \frac{1}{2} \int_{\Omega} \|\nabla f(x)\|^2 \, dx. \tag{1}$$

Intuitively, this quantity represents how much the function stretches the domain. If $f(x)$ is constant (flat), $\nabla f(x) = 0$ everywhere, and the energy is zero. Inversely, if $f(x)$ oscillates rapidly or has steep slopes, $\|\nabla f(x)\|^2$ is large, resulting in high energy.

In the context of Graph Neural Networks, the domain $\Omega$ is replaced by the set of nodes $V$, and the function $f$ is replaced by the scalar scores $s \in \mathbb{R}^N$. The gradient is approximated by the difference between values at connected nodes. For a graph with edge set $E$, the discrete Dirichlet energy is given by:

$$E_{\text{graph}}[s] = \sum_{(i,j) \in E} (s_i - s_j)^2. \tag{2}$$

This is related to the graph Laplacian quadratic form. If $L$ is the graph Laplacian matrix and $s$ is the vector of scores, then $s^\top L s = \sum_{(i,j)\in E} (s_i - s_j)^2$.

In `ManifoldRanker`, the code implements this concept directly in the training loop as the `geo_loss` (lines 799–800):

```
# Loss 4: Geometric Loss (Manifold Smoothness)
geo_loss = torch.mean((scores[src] - scores[dst])**2)
```

By minimizing this term (weighted by $\lambda_{\text{geo}} = 0.01$), the model penalizes high-frequency variations where neighboring municipalities have drastically different scores. This acts as a regularizer, based on the homophily assumption, i.e., connected nodes in the graph (which are similar in feature space) should likely have similar utility scores.

The use of Dirichlet energy minimization involves the critical trade-off between smoothing and oversmoothing. Reducing noise and propagating labels from high-confidence nodes to their neighbors is wanted, but, if the energy is minimized too aggressively (e.g., via too many GNN layers or a very high $\lambda_{\text{geo}}$), the system reaches a trivial solution where $s_i = s_j$ for all $i, j$. The system loses its discriminative power, and all municipalities receive the same rank. The `ManifoldRanker` balances this by using a low weight ($\lambda_{\text{geo}} = 0.01$) and relying on the residual connections in `ResGCNLayer` to preserve node-specific identities.

### 6.5 Total loss and weighting

The total loss is a linear combination (lines 802–806):

$$L = \lambda_{\text{rank}} L_{\text{rank}} + \lambda_{\text{recon}} L_{\text{recon}} + \lambda_{\text{geo}} L_{\text{geo}} + \lambda_{\text{compass}} L_{\text{compass}}.$$

The hyperparameters $\lambda$ are defined in `Config` (lines 133–146). They control trade-offs:

- Increasing $\lambda_{\text{rank}}$ forces stronger consistency with pairwise proxy orderings.
- Increasing $\lambda_{\text{compass}}$ forces scores to approximate the scalar proxy directly.
- Increasing $\lambda_{\text{recon}}$ makes embeddings preserve feature information more strongly.
- Increasing $\lambda_{\text{geo}}$ makes scores smoother along the graph, potentially reducing sensitivity to local noise but also potentially oversmoothing real discontinuities.

# 7 VRAM-awareness and optimization strategy

A central design goal is to avoid GPU out-of-memory errors while still training a GNN on a potentially large graph. The code combines some techniques in order to achieve this. One of them is to use CPU-side graph indexing with subgraph sampling (`SafeGraphEngine.sample_safe_subgraph`, lines 374–462). Another one is to use pinned CPU memory with non-blocking transfers (lines 703–712 and 755–759). Finally, gradient accumulation simulates larger batch sizes (lines 811–815).

The tensors `X_tens_cpu` and `X_par_tens_cpu` are created on the CPU and pinned using `pin_memory()` (lines 711–712). On CUDA systems, pinned (page-locked) host memory enables faster Direct Memory Acess (DMA) transfers and allows `.to(device, non_blocking=True)` to overlap copies with GPU computation when used with an appropriate CUDA stream. The code leverages this by keeping the full dataset in pinned RAM and transferring only sampled subgraphs per step (lines 755–758).

Training on a full $k$NN graph would require holding all node embeddings and all edges in GPU memory. Instead, each iteration samples a batch of center nodes and expands a local neighborhood on the CPU. This yields an induced subgraph with at most a few thousand nodes (the code enforces a hard cap of 5000 nodes, lines 418–426). Only this subgraph is transferred to the GPU.

Expansion is done by a breadth-first search over the $k$NN neighbor lists. At each hop, neighbor lists are randomly down-sampled to `MAX_NEIGHBORS_SAMPLE` (line 410). This reduces the theoretical growth from $O(k^\ell)$ to $O(s^\ell)$ where $s$ is the sample limit and $\ell$ is the number of layers/hops.

Gradient accumulation (lines 811–815) accumulates gradients across multiple forward/backward passes before applying an optimizer step. This effectively increases the batch size without increasing the memory footprint of any single subgraph.

**Attention:** When accumulating gradients over $K$ steps, each step's loss is divided by $K$ so that the accumulated gradient matches the gradient of a single large batch. The current code does not divide `total` by `GRAD_ACCUM_STEPS` before backpropagation, so the effective gradient magnitude is scaled by $K$. This is not necessarily wrong if the learning rate is tuned accordingly, but it does change the optimization dynamics.

The code clips the global gradient norm to 1.0 before each optimizer step (line 822). This is a stabilization technique for GNNs and recurrent architectures, where gradient explosion can occur due to repeated message passing. The code also calls `torch.cuda.empty_cache()` periodically (lines 841 and 869–870). This does not free memory held by tensors, but it releases unused cached blocks back to the CUDA allocator, which can reduce fragmentation in long-running jobs.

# 8 Code walkthrough

This section follows the code sequentially and explains the purpose of every block. Line numbers refer to Appendix A.

## 8.1 Imports and global utilities

Lines 1–4: Standard-library imports.

- `gc`: explicit garbage collection to release large objects early (lines 245–246).

Lines 6–7: Data and array libraries.

- `numpy`: vectorized operations.

Lines 9–12: PyTorch.

- `torch`: tensor operations on CPU/GPU.
- `torch.nn`, `torch.nn.functional`: layers and activation/loss functions.
- `torch.optim`: optimizers (AdamW, for instance).

Lines 14–18: Scikit-learn and SciPy utilities.

- Scalers: `StandardScaler`, `MinMaxScaler`, `RobustScaler` for normalization.
- `NearestNeighbors`: CPU-side $k$NN index for graph construction.
- `SpectralEmbedding`: Laplacian-eigenmap positional encodings.
- `PCA`: fallback positional encoding if spectral embedding is infeasible.
- `spearmanr`: rank correlation to validate that the learned ranking aligns with the proxy.

## 8.2 Design notes and stated limitations

Lines 21–33 state three research directions:

1. Replace GraphSAGE-like sampling with Graph Attention Networks (GATs), possibly asymmetric to respect partial ordering semantics.
2. Add evaluation visualizations (Brazil map, SHAP/feature-importance plots).
3. Study and visualize the manifold structure underlying the partial orders.

## 8.3 Configuration class `Config`

The `Config` class (lines 40–159) is a container of hyperparameters and global settings. All attributes are class-level constants, which makes them accessible as `Config.X` without instantiation.

### 8.3.1 Hardware selection and reproducibility

- `DEVICE` (line 61): selects CUDA if available, else CPU. This is used throughout via `to(Config.DEVICE)`.
- `SEED` (line 65) and `set_seeds` (lines 151–159): seeds NumPy and PyTorch RNGs. Note: full determinism may also require setting `torch.backends.cudnn.deterministic=True` and disabling benchmark mode.

### 8.3.2 Dataset configuration

- `CSV_FILE_PATH` (line 71): expected input file name.
- `ID_COL` (line 74): intended name of the unique municipality identifier column. The loader contains a fallback fuzzy match if this exact string is not found (lines 209–214).

#### 8.3.3 Neural architecture dimensions

- `HIDDEN_DIM` (line 81): width of the GNN hidden state.
- `LATENT_DIM` (line 85): bottleneck dimension inside the rank head. It is important to note that this is not the final score dimension, which is 1.
- `NUM_LAYERS` (line 108): number of message-passing layers. More layers increase receptive field but risk oversmoothing and increase sampled-subgraph size.
- `POS_ENC_DIM` (line 113): dimensionality of the positional encoding concatenated to the input features.

#### 8.3.4 Graph topology and VRAM constraints

- `KNN_K` (line 94): number of neighbors per node in the CPU-side $k$NN index. Higher $k$ improves connectivity but increases edge density.
- `MAX_NEIGHBORS_SAMPLE` (line 104): down-sampling limit during BFS subgraph expansion. This is critical, because it controls exponential neighborhood growth and, hence, GPU memory usage.

#### 8.3.5 Optimization hyperparameters

- `BATCH_SIZE` (line 120): number of center nodes per iteration.
- `GRAD_ACCUM_STEPS` (line 124): number of iterations to accumulate gradients before an optimizer step.
- `ITERATIONS` (line 126): total training steps.
- `LR` (line 127) and `WEIGHT_DECAY` (line 128): AdamW hyperparameters.

#### 8.3.6 Loss weights

Lines 133–146 define four coefficients that combine distinct training objectives. Changing these values changes what the optimizer seeks. The margin for the pairwise ranking loss is `MARGIN` (line 149).

### 8.4 Data engine `MunicipalityManifold`

`MunicipalityManifold` (lines 164–286) implements CSV ingestion and feature preprocessing. It is designed to minimize peak RAM usage and to produce two NumPy arrays: `X_model` for learning and `X_pareto` for proxy supervision.

In `__init__` (lines 174–186), a list of 19 feature column names are hard-coded(lines 180–186) in order to reduce memory footprint and avoid pre-processing ambiguity.

`load` (lines 188–248) implements a two-stage CSV access pattern that consists of column discovery, in which a one-row chunk (`chunksize=1`, line 206) is read to retrieve the actual column names without loading the full file and a selective load is executed via calling `pd.read_csv` with `usecols` (lines 218–219) so only the ID column and the selected features are materialized in RAM. This is a standard technique for large CSVs: filtering at read time is cheaper than reading everything and dropping columns afterward.

The intended ID column name is `Config.ID_COL` (line 74). If this string is not found in the file, the code searches the column names for substrings such as `'Ibge'` or `'Munic'` (lines 209–214). If a match is found, the dataframe is renamed to standardize on `Config.ID_COL` (lines 221–223).

After loading, the code enforces numeric types for each feature column (lines 229–234). In particular, two edge cases are handled. If a feature column is missing, it is created and filled with zeros (line 232). If a column contains non-numeric strings, `pd.to_numeric(..., errors='coerce')` converts them to NaN, which are then filled with 0.0 (line 233).

**Attention:** The choice of imputing missing/invalid values as 0 is a strong modeling assumption. It implicitly treats missing as "no infrastructure" rather than "unknown". If missingness is informative or systematic, alternative imputations (median, learned embeddings, missingness indicators) may be more appropriate.

Lines 235–246 extract the ID vector and the raw feature matrix as NumPy arrays. The features are cast to `float32` (line 239) to halve memory relative to the default `float64`. Immediately after extraction, the dataframe is deleted and garbage collection is forced (lines 245–246). This reduces peak RAM usage during the scaling steps.

`_process_features` (lines 250–286) implements the three-stage normalization described earlier (Log1p → RobustScaler → StandardScaler) and then constructs the Min–Max proxy. The method returns (`X_model`, `X_pareto`, `ids`) (line 286).

### 8.5 Graph topology `SafeGraphEngine`

`SafeGraphEngine` (lines 291–462) is responsible for (i) building a reusable CPU-side neighbor index, (ii) computing positional encodings, and (iii) sampling induced subgraphs for training and inference.

The constructor `__init__` (lines 310–372) receives the full feature matrix $X^{\text{model}}$ and a neighbor count $k$ (default 15). It stores the number of nodes `self.N` (line 320), the number of neighbors stored per node `self.k` (line 321), and the per-hop neighbor down-sampling limit `self.sample_limit` (line 324) used during subgraph expansion.

Nearest-neighbor search is configured with `algorithm='kd_tree'` and `n_jobs=-1` (line 332). A KD-tree provides sub-quadratic neighbor search in moderate dimensions; performance degrades in very high dimensions due to the curse of dimensionality, but with $D = 19$ it is most likely still effective.

The full neighbor index matrix `self.indices` is computed once via `kneighbors` (line 337) and stored in CPU RAM. This is important, because it allows fast subgraph sampling without recomputing distances or storing a global adjacency matrix on GPU.

Lines 339–372 compute `self.pos_enc`. Jittering (line 342) adds tiny Gaussian noise to avoid numerical issues when points are identical (spectral methods can be sensitive to exact duplicates). Method selection (lines 356–369) tries to use as the positional ecoding spectral embedding, unless $N > 15000$, when PCA is used. Standardization (line 372) rescales positional coordinates to comparable magnitude across dimensions.

The method `sample_safe_subgraph` (lines 374–462) is an important VRAM-usage mechanism. It takes a set of center node indices (global IDs) and expands an induced subgraph around them up to `num_layers` hops.

**Step 1: BFS expansion with neighbor down-sampling.** The method maintains a set `nodes` and a current `frontier` (lines 399–401). For each hop (line 403):

1. Each frontier node's full neighbor list is retrieved from `self.indices` (line 405).

2. If the neighbor list length exceeds `sample_limit`, a subset of columns is randomly chosen (lines 410–413).
3. The sampled neighbors are flattened to get `next_frontier` (line 416).
4. A hard cap of 5000 total nodes is enforced (lines 418–426).
5. `next_frontier` is added to the node set and the process is repeated.

**Step 2: Create an induced edge list.** After expansion, `nodes` is converted to a sorted NumPy array `all_nodes` (line 432). The code then constructs all outgoing edges from these nodes according to the original $k$NN index (lines 438–440):

- `src_global`: repeats each node ID $k$ times.
- `dst_global`: concatenates the $k$ neighbor IDs for each node.

Edges are filtered to keep only those whose destinations are also inside the sampled node set (lines 442–448). Specifically, this is the definition of an induced subgraph.

**Step 3: Remap global IDs to local IDs.** GPU computation expects node indices $0, \ldots, n-1$ for a subgraph of size $n$. Because `all_nodes` is sorted, `np.searchsorted` can map each global ID to its local index efficiently (lines 451–453). The method returns`all_nodes` as a tensor of global node IDs (line 462 return), `edge_index` as a `[2, num_edges]` tensor of local edge endpoints (line 456), and `batch_idx` mapping the original center nodes to local indices so losses can be computed on center nodes only (lines 458–460).

### 8.6 Graph layer `ResGCNLayer`

`ResGCNLayer` (lines 468–544) is a custom implementation of a mean-aggregating graph convolution with a residual connection. The constructor in `__init__` (lines 479–487) contains `self.proj`, which is a learnable linear map $W_{\text{proj}}$ that transforms input features before aggregation (line 483), and `self.skip`, which can either be identity (if dimensions match) or a linear projection to align dimensions for residual addition (line 487).

**Forward pass.** The forward method (lines 489–544) implements sparse message passing without materializing a dense adjacency matrix:

1. If there are no edges, then only the skip projection is returned (lines 496–499).
2. `row` and `col` from `edge_index` are read (line 501).
3. Projected node features `h = self.proj(x)` are computed (line 505).
4. Source features are gathered per edge: `h_src = h[row]` (line 514).
5. Scatter-add into destinations: `out.index_add_(0, col, h_src)` (line 521).
6. In-degrees are computed with a second scatter-add (lines 526–530).
7. Normalization by degree is performed to obtain mean aggregation (line 532).

8. The residual projection `self.skip(x)` is added and the GELU activation function is applied (line 544).

The scatter-add operation is the computational core of the sparse message-passing mechanism used in `ResGCNLayer`. It allows the model to aggregate information from neighbors without materializing a dense adjacency matrix, which is the primary enabler of the VRAM-efficient design.

In standard grid-based deep learning (e.g., Convolutional Neural Networks for images), every pixel has a fixed number of neighbors, allowing for efficient matrix multiplication. Graphs, however, are irregular: one municipality may have 2 neighbors while another has 20. A naive implementation would use a dense adjacency matrix $A \in \{0,1\}^{N \times N}$. For $N = 5570$, this matrix contains over 31 million entries, most of which are zero. Storing and multiplying this matrix on the GPU is memory-inefficient.

Scatter-add operates on a list of edges rather than a matrix. Given a coordinate list of edges (COO format) where vectors `row` and `col` represent source and destination indices respectively, the operation performs (1) gathering, that is, for every edge $k$, retrieving the feature vector $h_{\text{row}[k]}$ of the source node, (2) scattering, i.e., mapping these vectors to their corresponding destination indices $\text{col}[k]$, and accumulating (summing) the vectors into the destination slots. Mathematically, for a destination node $i$, the output is

$$\text{out}_i = \sum_{k:\text{col}[k]=i} h_{\text{row}[k]}, \tag{3}$$

which, in code, is implemented as in lines 514–521:

```
# Line 501: Unpack edge indices
row, col = edge_index

# Line 514: Gather source features
# Creates a tensor of shape [Num_Edges, Hidden_Dim]
h_src = h[row]

# Line 521: Scatter-add into destinations
# out[col[k]] += h_src[k]
out.index_add_(0, col, h_src)
```

Let there be three edges pointing to two destination cities (X and Y). The scatter-add operation accumulates features from incoming neighbors (A and B) into City X, while City Y receives features from City C.

Table 2: Trace of scatter-add operation for a small subgraph.

| **Edge** | **Source (row)** | **Dest (col)** | **Effect on Accumulator (`out`)** |
|---|---|---|---|
| 1 | City A ($h = [0.1, 0.5]$) | City X | $\text{out}_X \leftarrow [0.0, 0.0] + [0.1, 0.5]$ |
| 2 | City B ($h = [0.2, 0.1]$) | City X | $\text{out}_X \leftarrow [0.1, 0.5] + [0.2, 0.1] = [\mathbf{0.3}, \mathbf{0.6}]$ |
| 3 | City C ($h = [0.9, 0.9]$) | City Y | $\text{out}_Y \leftarrow [0.0, 0.0] + [0.9, 0.9] = [\mathbf{0.9}, \mathbf{0.9}]$ |

By processing only the $O(|E|)$ existing edges rather than the $O(N^2)$ potential connections, the scatter-add operation reduces VRAM usage significantly. From a graph signal processing viewpoint, this layer applies a learned linear transform and then averages neighbor messages, which is analogous to a learned, low-pass smoothing operator. The residual path counteracts excessive smoothing.

### 8.7 Model `ManifoldRanker`

`ManifoldRanker` (lines 546–668) is the full GNN model with two heads. It consumes (i) standardized features, (ii) positional encodings, and (iii) a subgraph edge list. The model concatenates each node's domain features and positional encoding (lines 562–566 and 623–626):

$$z_i^{(0)} = \mathrm{GELU}\big(W_{\mathrm{in}}\,[x_i^{\mathrm{model}}; p_i] + b_{\mathrm{in}}\big)\,,$$

where $[\,\cdot;\cdot\,]$ denotes concatenation. This produces a hidden representation in $\mathbb{R}^{\texttt{HIDDEN_DIM}}$.

`self.layers` is a list of `NUM_LAYERS` identical-width `ResGCNLayer`s (lines 568–571). Each layer updates node representations using the subgraph connectivity. After the stack, a `LayerNorm` is applied (line 632). Layer normalization stabilizes optimization by normalizing activations per node.

The ranking head `rank_mlp` (lines 603–607) computes an intermediate latent vector and then a scalar: $z_i = \tanh(W_1 h_i + b_1)$ and $a_i = W_2 z_i + b_2$. The final score is `torch.sigmoid(a_i)` (line 635), ensuring $s_i \in [0, 1]$. The decoder `self.decoder` (lines 585–589) maps $h_i$ back to feature space via a two-layer MLP with ELU non-linearity. It outputs $\hat{x}_i \in \mathbb{R}^D$ without bounding; the reconstruction loss compares it directly to the standardized input features. `enforce_monotonicity` (lines 640–668) clamps the weights of the final linear layer of `rank_mlp` to be non-negative (line 668). This guarantees that, holding the latent vector $z_i$ fixed, the score is non-decreasing in each component of $z_i$ because

$$\frac{\partial s_i}{\partial z_{i,j}} = \sigma'(a_i)\, w_j \geq 0 \quad \text{if } w_j \geq 0.$$

This constraint does not guarantee monotonicity with respect to the original input features $x_i$ because earlier layers are unconstrained and can map increases in an input feature to decreases in some latent component. Achieving end-to-end monotonicity typically requires constraining all relevant weights (and using monotone activations) throughout the network.

### 8.8 Execution `main`

The `main` function (lines 674–895) performs seed setup, data loading, pinned-memory preparation, topology construction, training, inference, and result export.

Lines 711–712 create pinned CPU tensors from NumPy arrays. These tensors represent the entire dataset and remain in host memory. Only subgraphs are gathered and transferred to the GPU per iteration. Line 716 constructs `SafeGraphEngine`, which precomputes `self.indices` and `self.pos_enc`. The positional encodings are also converted to a pinned CPU tensor (line 719). The model is instantiated and moved to the chosen device (line 723). Only model parameters live persistently in GPU VRAM. Optimization uses AdamW (line 724), which is Adam with decoupled weight decay.

**Training loop** The core training loop runs for `ITERATIONS` steps (lines 737–842). Each iteration performs the following set of steps.

1. Samples `BATCH_SIZE` random center nodes (line 742). This is an instance of uniform random sampling with replacement.

2. Builds a local induced subgraph around these centers using `sample_safe_subgraph` (line 750).

3. Gathers features, positional encodings, and proxy features for the subgraph from pinned memory and transfers them to GPU (lines 755–759).

4. Runs the model on the subgraph (line 763) to obtain scores and reconstructions for all subgraph nodes.
5. Selects the scores for center nodes only and computes the four losses (lines 768–800).
6. Combines losses and back-propagates (lines 802–809).
7. Every `GRAD_ACCUM_STEPS` steps, clips gradients, updates parameters, clamps monotonicity, and clears gradients (lines 815–832).
8. Logs diagnostics and clears CUDA cache periodically (lines 834–842).

After training, the code computes scores for all nodes in micro-batches of size 512 (lines 843–871). For each chunk of nodes, it constructs a 1-hop subgraph (line 855), transfers the relevant tensors, runs the model, and stores the resulting scores back into a NumPy array (line 864). The model has `NUM_LAYERS` message-passing layers (default 2), but in the inference step subgraphs are sampled using only 1 hop (`num_layers=1`, line 855). This means the second GNN layer cannot access true 2-hop neighbors not included in the sampled node set. This is an intentional speed/memory trade-off, but it changes the effective receptive field relative to training.

Finally, the code reloads the original CSV (line 874), attaches the utility scores and ranks (lines 880–882), and computes Spearman correlation between the learned scores and the proxy mean (lines 889–891). Spearman correlation measures rank agreement. The results are saved to `Final_Rankings.csv` (line 893).

## 9 Critical observations and potential improvements

As discussed in the loss section, the masking operation in line 792 does not behave as intended because `MarginRankingLoss` defaults to reduction `'mean'`. A correct masked implementation would probably conceptually be

```
margin_loss_fn = nn.MarginRankingLoss(margin=Config.MARGIN, reduction='none')
pair_loss = margin_loss_fn(s_A, s_B, target) # shape [B]
rank_loss = (pair_loss * mask).sum() / (mask.sum().clamp(min=1.0))
```

This change ensures that pairs with `mask=0` contribute neither loss nor gradient.

`target = torch.sign(u_A - u_B)` (line 787) can produce zeros when $u_A = u_B$. If such pairs are not masked out before calling the margin loss, they would contribute a constant margin term with zero gradient. The code attempts to mask small differences (line 791), which includes ties, but due to the reduction issue, ties still affect the reduced loss. I estimate proper per-pair masking is necessary to handle ties.

Without dividing `total` by `GRAD_ACCUM_STEPS`, accumulated gradients are larger by approximately that factor. To simulate a larger batch size faithfully, I believe the adjustment

```
(total / Config.GRAD_ACCUM_STEPS).backward()
```

is necessary. The necessity of this approach depends on how the learning rate is tuned; the current code is still effective, but the interpretation of `LR` is not the standard one from conventional theory.

It is important to note that, since the $k$NN graph is directed, the neighborhood aggregation is not symmetric. Some nodes can become hubs with high in-degree even if they have fixed out-degree $k$. This can potentially interact with degree normalization and with the geometric loss. If an undirected similarity graph is desired, the edge set can be symmetrized (e.g., include $(i, j)$ if either $i$ is in $k\mathrm{NN}(j)$ or $j$ is in $k\mathrm{NN}(i)$).

If `NearestNeighbors.kneighbors` returns each point as its own neighbor, then self-loops are present. In that case, each node contributes to itself via (i) the aggregation path and (ii) the explicit skip connection. It is still unclear what the effects of this mechanism are.

The process of inference uses `num_layers=1` for subgraph sampling (line 855) even though the model may have 2 message-passing layers. This reduces computation but changes the information available to deeper layers. We may have to use an inference sample using the same hop count as training for the sake of consistency between training and inference.

The current supervision signal is based on the mean of Min–Max scaled features, which is a particular type of scalarization. In order to respect Pareto dominance more directly, it is possible that it may be necessary to construct pairwise constraints based on dominance: for instance, for any $(A, B)$ such that $A$ dominates $B$, enforce $s_A \geq s_B + m$. I believe this would retain the partial-order structure rather than imposing a single scalar proxy.

Clamping only the final layer weights guarantees monotonicity with respect to the last latent coordinates but not with respect to original features. If the domain knowledge requires monotonicity with respect to specific raw indicators, monotone-network constructions (constraining weights in all layers, using monotone activations, or using specialized monotone architectures) may be required. I already pointed out to this direction in lines 655–663. However, I am afraid that doing so might occasion loss of generality.

# A Source Code

The listing below is the full Python source code.

```
import os
import time
import warnings
import gc

import numpy as np
import pandas as pd

import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim

from sklearn.preprocessing import StandardScaler, MinMaxScaler, RobustScaler
from sklearn.neighbors import NearestNeighbors
from sklearn.manifold import SpectralEmbedding
from sklearn.decomposition import PCA
from scipy.stats import spearmanr


# Problems that I see with the current code:

# 1) I do not like the use of GraphSAGE. I wanted to used Graph Attention Networks (GATs)
    ,
# because we could produce something innovative, such as asymetric, skewed GATs,
    respecting
# the partial ordering of ranking analyses.

```

```
# 2) We need visualizations and graphs to check how the ranking is performing:
# a - I believe we need to have a map of Brazil showing grades per municipality.
# b - Also, it would be useful to have SHAP analyses for feature importance, including
# Beeswarm matrices.

# 3) I would like to undersstand the manifold on which these partial orderings sit.
# Can we plot the manifold? How could it be understood?

# ==============================================================================
# CONFIGURATION
# ==============================================================================
warnings.filterwarnings("ignore")

class Config:
    """
    Configuration for the ranker.

    This class acts as a central control panel for the model. It is designed
    specifically for VRAM safety, meaning it prioritizes memory efficiency
    to prevent GPU Out-Of-Memory (OOM) errors during the training of
    Graph Neural Networks (GNNs).

    Key Concepts Managed Here:
        1. Hardware Acceleration (Device detection)
        2. Neural Architecture (Dimensions, Graph Topology)
        3. Memory Optimization (Neighbor Sampling, Gradient Accumulation)
        4. Multi-Objective Loss Balancing (Ranking vs. Geometry)
    """

    # ------------------------------------------------------------------------
    # HARDWARE
    # ------------------------------------------------------------------------
    # Automatically detect if an NVIDIA GPU is available.
    # If 'cuda' is present, computations are moved to the GPU for speed.
    DEVICE: torch.device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    # Setting this ensures that random initialization of weights
    # and data shuffling is identical every time you run the script.
    SEED: int = 42

    # ------------------------------------------------------------------------
    # 2. DATASET SPECIFICATIONS
    # ------------------------------------------------------------------------
    # Path to the pre-processed feature file.
    CSV_FILE_PATH: str = 'input_dataset_with_indicators.csv'

    # The unique identifier column for each entity (e.g., municipality code).
    ID_COL: str = 'Código IBGE'

    # ------------------------------------------------------------------------
    # NEURAL ARCHITECTURE
    # ------------------------------------------------------------------------
    # HIDDEN_DIM: Width of internal layers.
    # Larger = more capacity, but higher memory cost.
```

```
    HIDDEN_DIM: int = 64

    # LATENT_DIM: Size of the final compressed embedding.
    # This is the "manifold" space where we measure distances and ranking.
    LATENT_DIM: int = 32

    # ---------------------------------------------------------------------------
    # GRAPH TOPOLOGY AND VRAM SAFETY
    # ---------------------------------------------------------------------------
    # KNN_K: Pool of Potential Connections.
    # We set this to KNN_K to ensure the graph forms a single connected component.
    # If k is too low (e.g., 5) at this stage, the graph might break into
    # disconnected 'islands', preventing global manifold learning.
    KNN_K: int = 15

    # MAX_NEIGHBORS_SAMPLE: 'Active Training Set'.
    # Even though a node has k connections (KNN_K), processing all of them
    # simultaneously requires too much GPU memory.
    #
    # We sample 5 random neighbors per step. This acts as 'Edge Dropout':
    # 1. Reduces VRAM usage.
    # 2. Prevents overfitting (Regularization).
    # 3. Over many epochs, the model still sees all k neighbors.
    MAX_NEIGHBORS_SAMPLE: int = 5

    # NUM_LAYERS: Depth of the GNN (message-passing hops).
    # Kept low to prevent "Oversmoothing" and exponential node growth.
    NUM_LAYERS: int = 2

    # POS_ENC_DIM: Positional Encoding Dimension.
    # Adds coordinate information (via Spectral Embedding or PCA) so the GNN
    # knows "where" a node sits in the global cluster structure.
    POS_ENC_DIM: int = 8

    # ---------------------------------------------------------------------------
    # OPTIMIZATION AND TRAINING LOOP
    # ---------------------------------------------------------------------------
    # BATCH_SIZE: How many center nodes we process per step.
    # Kept small to fit into GPU memory.
    BATCH_SIZE: int = 128

    # GRAD_ACCUM_STEPS: Virtual Batch Size Trick
    # We accumulate gradients over GRAD_ACCUM_STEPS batches and update once.
    GRAD_ACCUM_STEPS: int = 4

    ITERATIONS: int = 2000
    LR: float = 1e-3
    WEIGHT_DECAY: float = 1e-4

    # ---------------------------------------------------------------------------
    # MULTI-OBJECTIVE LOSS FUNCTION
    # ---------------------------------------------------------------------------
    # LAMBDA_RANK: Enforces correct sorting order (Learning to Rank).
    LAMBDA_RANK: float = 1.0
```

```python

    # LAMBDA_RECON: Autoencoder loss.
    # Prevents the model from "cheating" by ensuring embeddings still represent input data.
    LAMBDA_RECON: float = 0.1

    # LAMBDA_GEO: Manifold loss.
    # Penalizes if neighbors in input space become far apart in latent space.
    LAMBDA_GEO: float = 0.01

    # LAMBDA_COMPASS: Directional bias.
    # Guides the latent space so that "better" metrics move in a positive direction.
    LAMBDA_COMPASS: float = 1.0

    # MARGIN: The safety buffer required between two items in ranking loss.
    MARGIN: float = 0.1

    @staticmethod
    def set_seeds() -> None:
        """
        Enforces deterministic behavior across NumPy, PyTorch CPU, and PyTorch CUDA.
        """
        np.random.seed(Config.SEED)
        torch.manual_seed(Config.SEED)
        if torch.cuda.is_available():
            torch.cuda.manual_seed_all(Config.SEED)

# ==============================================================================
# DATA ENGINE
# ==============================================================================
class MunicipalityManifold:
    """
    This class handles the transition from raw CSV storage to normalized
    tensors ready for the Neural Network.

    By strictly enforcing type casting (float32) and aggressive garbage collection
    (gc.collect), this class ensures that the system RAM is not bloated.
    This is vital because we rely on 'Pinned Memory' (System RAM that the GPU
    can access quickly) to feed the GPU without crashing it.
    """
    def __init__(self, filepath):
        self.filepath = filepath
        # We manually specify the exact columns required for the analysis.
        # This prevents the dataframe from loading unnecessary metadata columns
        # that would waste memory.
        # These columns represent Telecommunications Infrastructure indicators
        self.feature_cols = [
            'Cobertura_5G_Moradores', '5G', 'Ibc', 'Isg',
            'Resultado_IND1', 'Resultado_IND2', 'Resultado_IND3', 'Resultado_IND4',
            'Resultado_IND5', 'Resultado_IND6', 'Resultado_IND7', 'Resultado_IND8',
            'Resultado_INF4-DL-3G', 'Resultado_INF4-DL-4G', 'Resultado_INF4-DL-5G',
            'Resultado_INF4-UP-3G', 'Resultado_INF4-UP-4G', 'Resultado_INF4-UP-5G'
        ]

```

```python
    def load(self):
        """
        Reads the CSV with memory-optimized strategies.

        Returns:
            X_model (np.array): normalized features for the Neural Network.
            X_pareto (np.array): 0-1 scaled features for the 'Compass' Loss.
            ids (np.array): The identifiers (IBGE Codes) to track the municipality.
        """
        print(f"--- [Data] Loading: {self.filepath} ---")
        if not os.path.exists(self.filepath):
            raise FileNotFoundError(f"File {self.filepath} not found.")

        try:
            # CSVs often have slightly different column names
            # (e.g., 'Cod_IBGE' vs 'Código IBGE').
            # We read just the first chunk (header) to inspect columns
            # without loading the entire multi-megabyte file into memory.
            df_iter = pd.read_csv(self.filepath, sep=';', encoding='utf-8-sig', chunksize=1)
            actual_cols = next(df_iter).columns.tolist()

            # Fuzzy match the ID column if the exact name is not found
            id_col = Config.ID_COL
            if id_col not in actual_cols:
                for c in actual_cols:
                    if 'Ibge' in c or 'Munic' in c: id_col = c; break

            # Memory-Optimized Load
            # We use 'usecols' to filter data before it hits RAM.
            # Pandas only constructs the DataFrame with the requested columns.
            df = pd.read_csv(self.filepath, sep=';', encoding='utf-8-sig', decimal='.',
                             usecols=lambda c: c in self.feature_cols or c == id_col)

            # Standardize the ID column name for downstream consistency
            if id_col != Config.ID_COL:
                df.rename(columns={id_col: Config.ID_COL}, inplace=True)

        except Exception as e:
            print(f"Error reading CSV: {e}")
            return None, None, None

        # We force conversion to numeric types and treat errors (e.g. text
        # strings in number fields) as Missing/NaN, then fill with 0.0.
        for col in self.feature_cols:
            if col not in df.columns: df[col] = 0.0
            df[col] = pd.to_numeric(df[col], errors='coerce').fillna(0.0)

        # Extract numpy arrays
        ids = df[Config.ID_COL].values
        # Ensure float32 (default pandas is float64).
        # float32 cuts memory usage in half (crucial for large matrices).
        X_raw = df[self.feature_cols].values.astype(np.float32)

```

```
        # Explicit Garbage Collection
        # Python keeps 'df' in memory until the function ends. Since 'df' is potentially large,
        # and we now have 'X_raw', we explicitly delete 'df' and force garbage collection
        # to free up RAM immediately for the upcoming scaling operations.
        del df
        gc.collect()

        return self._process_features(X_raw, ids)

    def _process_features(self, X_raw, ids):
        """
        Applies a normalization pipeline designed for Neural Networks.

        Infrastructure data is rarely Gaussian. It usually follows a Power Law.
        (e.g., São Paulo has 1000x the infrastructure of a small town).

        If we feed raw data to the Neural Network, gradients could explode or vanish.
        """
        print("Processing: Log1p -> RobustScaler -> StandardScaler")

        # Log1p (Logarithm + 1)
        # Compresses the "Long Tail". It brings massive values (1,000,000) closer
        # to small values (10) so the model can see differences in both.
        X_log = np.log1p(X_raw)

        # RobustScaler
        # Uses the Interquartile Range (IQR).
        # Unlike MinMax, this ignores extreme outliers. If one city has a weird glitch
        # value, it won't squash everyone else's data to zero.
        scaler_robust = RobustScaler(quantile_range=(5.0, 95.0))
        X_temp = scaler_robust.fit_transform(X_log)

        # StandardScaler
        # Centers the data (Mean = 0, Std = 1).
        # Neural Networks (and especially the Tanh/GELU activations we use later)
        # work best when inputs are centered around zero.
        scaler_std = StandardScaler()
        self.X_model = scaler_std.fit_transform(X_temp).astype(np.float32)

        # Pareto/MinMax Proxy
        # We need a simple score (0 to 1) to guide the 'Compass Loss'.
        # We use MinMax on the Log data to create a rough "better vs worse" heuristic.
        scaler_mm = MinMaxScaler()
        self.X_pareto = scaler_mm.fit_transform(X_log).astype(np.float32)

        return self.X_model, self.X_pareto, ids

# ==============================================================================
# GRAPH TOPOLOGY
# ==============================================================================
class SafeGraphEngine:
    """
    Manages the graph structure.
```

```

    This class implements a 'Graph Sampling' strategy (similar to GraphSAGE).

    ***
    ATTENTION:
    I think that we can optimize this to become a Graph Attention Network
    (GAT), which I think could be a better architecture for this network.
    ***

    Instead of passing a massive global Adjacency Matrix to the GPU, it:
    1. Indexes the data on the CPU.
    2. Samples small, localized subgraphs around specific training nodes.
    3. Streams these small subgraphs to the GPU for processing.

    This allows training on massive datasets using modest GPU hardware.
    """
    def __init__(self, X_data, k=15):
        """
        Initializes the graph topology engine.

        Args:
            X_data (np.ndarray): normalized feature matrix (N_samples, N_features).
            k (int): Number of nearest neighbors to connect.
                    chosen to ensure the graph is fully connected
                    (no isolated nodes), allowing information to flow globally.
        """
        self.N = X_data.shape[0]
        self.k = k
        # The strict limit on how many neighbors we actually use during training.
        # This prevents the exponential growth of the computational graph.
        self.sample_limit = Config.MAX_NEIGHBORS_SAMPLE

        # We index k neighbors here to guarantee graph connectivity.
        print(f"[Topology] Indexing Graph (k={k})")

        # KD-Tree Indexing
        # We use a KD-Tree (k-dimensional tree) for O(log N) neighbor lookup.
        # 'Brute force' distance calculation is O(N^2), which is too slow.
        self.nbrs = NearestNeighbors(n_neighbors=k, algorithm='kd_tree', n_jobs=-1).fit(
            X_data)

        # Pre-compute Indices (CPU RAM)
        # We store the neighbor indices in System RAM (CPU),
        # not on the GPU. We only move specific rows to the GPU when needed.
        _, self.indices = self.nbrs.kneighbors(X_data)

        print("--- [Topology] Computing Embeddings ---")
        # Add slight jitter (white noise) to prevent Singular Matrix errors
        # when data points are identical.
        X_jitter = X_data + np.random.normal(0, 1e-6, X_data.shape)

        # Positional Encoding Strategy
        # GNNs are permutation-invariant: they do not know up from down or left from right
            .
```

```
        # We need to give the model a coordinate system (Positional Encoding) so it
        # understands the global shape of the manifold.
        #
        # Method A: Spectral Embedding (Laplacian Eigenmaps)
        # - Captures complex, non-linear manifold geometry.
        # - Computationally expensive: O(N^3). Fails on large N (>15k).
        #
        # Method B: PCA (Principal Component Analysis)
        # - Linear approximation.
        # - Very fast and robust. Used as a fallback if Method A crashes.
        try:
            if self.N > 15000: raise Exception("Use PCA for large N to save time/memory")

            embedding = SpectralEmbedding(n_components=Config.POS_ENC_DIM,
                                          n_neighbors=k,
                                          affinity='nearest_neighbors',
                                          random_state=Config.SEED,
                                          n_jobs=-1)
            self.pos_enc = embedding.fit_transform(X_jitter).astype(np.float32)
        except:
            print("[Info] Using PCA for Positional Encoding.")
            # Fallback to PCA if Spectral Embedding fails (e.g., disconnected graph or OOM
                )
            pca = PCA(n_components=Config.POS_ENC_DIM)
            self.pos_enc = pca.fit_transform(X_jitter).astype(np.float32)

        # Normalize positional encodings so they do not overpower the actual features
        self.pos_enc = StandardScaler().fit_transform(self.pos_enc).astype(np.float32)

    def sample_safe_subgraph(self, center_nodes_idx, num_layers=2):
        """
        If we simply took all neighbors of neighbors for 2 layers with k=15:
        1 Node -> 15 Neighbors -> 225 Neighbors.
        In a batch of 128 nodes, that's 28,800 nodes to process.

        This function imposes a 'sample_limit' (e.g., 5):
        1 Node -> 5 Neighbors -> 25 Neighbors.
        Batch of 128 -> 3,200 nodes. (safer for GPU memory).

        In other words, 'sample_safe_subgraph' performs a form of stochastic
        neighborhood sampling (similar to GraphSAGE). This method bridges the
        gap between the static topology (e.g., k = 15) and the memory constraints
        (e.g., limit = 5).

        Args:
            center_nodes_idx (np.array): The Global IDs of the nodes in the current batch.
            num_layers (int): How many hops (degrees of separation) to expand.

        Returns:
            all_nodes (tensor): List of all unique nodes involved in this subgraph.
            edge_index (tensor): The connectivity matrix (Adjacency) remapped to
                                 local indices (0 to len(all_nodes)).
            batch_idx (tensor): The local indices of the original 'center_nodes'.
        """
```

```
        nodes = set(center_nodes_idx)
        frontier = center_nodes_idx

        # BFS Expansion (Breadth-First Search)
        for _ in range(num_layers):
            # Retrieve all neighbors of the current frontier from the CPU index
            potential_neighbors = self.indices[frontier] # Shape: (frontier_size, k)

            # Random Downsampling
            # If a node has too many neighbors, we randomly pick a subset.
            # This breaks the exponential growth curve.
            if potential_neighbors.shape[1] > self.sample_limit:
                # Pick 'sample_limit' random columns
                rand_cols = np.random.choice(potential_neighbors.shape[1], self.
                    sample_limit, replace=False)
                potential_neighbors = potential_neighbors[:, rand_cols]

            # Flatten to get a list of unique next-step nodes
            next_frontier = np.unique(potential_neighbors.flatten())

            # Safety Cap
            # Hard stop if subgraph gets massive (>5000 nodes).
            # This protects the training loop from crashing if a "Super Node" (hub) is hit
                .
            if len(nodes) + len(next_frontier) > 5000:
                needed = 5000 - len(nodes)
                if needed > 0:
                    next_frontier = next_frontier[:needed]
                    nodes.update(next_frontier)
                break

            nodes.update(next_frontier)
            frontier = next_frontier

        # Convert set to sorted array for deterministic indexing
        all_nodes = np.sort(np.array(list(nodes)))

        # Build Local Adjacency Matrix (Induced Subgraph)
        # The GNN on the GPU needs a matrix where indices range from 0 to Subgraph_Size.
        # It does NOT know about the Global ID, only about the Local ID.

        # Get all edges originating from the selected nodes
        src_global = np.repeat(all_nodes, self.k)
        dst_global = self.indices[all_nodes].flatten()

        # Filter Edges
        # We only keep an edge if BOTH the source and destination are in our subgraph.
        # This creates a closed system (Induced Subgraph).
        mask = np.isin(dst_global, all_nodes)

        valid_src = src_global[mask]
        valid_dst = dst_global[mask]

        # Map Global IDs -> Local IDs
```

```
        # np.searchsorted is a fast O(log N) binary search.
        local_src = np.searchsorted(all_nodes, valid_src)
        local_dst = np.searchsorted(all_nodes, valid_dst)

        # Create the PyTorch tensor for edge indices [2, Num_Edges]
        edge_index = torch.tensor([local_src, local_dst], dtype=torch.long)

        # Map the original Batch Centers to Local IDs
        # We need to know which nodes to calculate the loss on.
        batch_idx = torch.tensor(np.searchsorted(all_nodes, center_nodes_idx), dtype=torch
            .long)

        return torch.tensor(all_nodes, dtype=torch.long), edge_index, batch_idx


# ==============================================================================
# MODEL ARCHITECTURE
# ==============================================================================
class ResGCNLayer(nn.Module):
    """
    A Residual Graph Convolutional Layer (ResGCN).

    Standard GCNs often suffer from "Oversmoothing" (features becoming indistinguishable)
    as they get deeper. This layer solves that by adding a Residual (Skip) Connection,
    similar to ResNet in Computer Vision.

    Formula:
        h_new = Activation( Neighborhood_Agg(h) + Linear(h) )
    """
    def __init__(self, in_dim, out_dim):
        super().__init__()

        # The main transformation weight matrix
        self.proj = nn.Linear(in_dim, out_dim)

        # The skip connection. If input/output dimensions differ, we need a
        # linear projection to match them. Otherwise, it is just a pass-through (Identity)
            .
        self.skip = nn.Linear(in_dim, out_dim) if in_dim != out_dim else nn.Identity()

    def forward(self, x, edge_index):
        """
        Args:
            x (Tensor): Node features [Num_Nodes, In_Dim]
            edge_index (Tensor): Graph connectivity in COO format [2, Num_Edges]
                                 row = source indices, col = destination indices.
        """
        # Edge Case: If a sampled subgraph has 0 edges (single isolated node),
        # we skip convolution and just return the residual.
        if edge_index.numel() == 0:
            return self.skip(x)

        row, col = edge_index

```

```
        # Linear Projection
        # Transform features before aggregation (Standard GCN practice)
        h = self.proj(x)

        # MANUAL MESSAGE PASSING
        # Instead of creating a dense Adjacency Matrix (N x N) which explodes VRAM,
        # we perform "Scatter-Gather" operations on the sparse edge list.

        # Gather Source Messages
        # For every edge, grab the feature vector of the source node.
        # Shape: [Num_Edges, Hidden_Dim]
        h_src = h[row]

        # Aggregate into Destination
        # We initialize an empty canvas (out) and add the source features
        # to their respective destination nodes (col).
        # This is equivalent to Sum-Pooling neighbors.
        out = torch.zeros_like(h)
        out.index_add_(0, col, h_src)

        # Degree Normalization
        # Popular nodes (hubs) receive massive sums, while isolated nodes get small ones.
        # So, we divide the sum by the node's degree (Mean-Pooling).
        deg = torch.zeros(x.size(0), 1, device=x.device)

        # Count how many incoming edges each node has
        deg.index_add_(0, col, torch.ones(row.size(0), 1, device=x.device))

        # Avoid division by zero with epsilon (1e-6)
        out = out / (deg + 1e-6)

        # Activation and Residual
        # We add the original input (self.skip(x)) to the new aggregation (out)
        # before applying the non-linearity (GELU).

        # ***
        # ATTENTION:
        # We have to check if the GELU is a good activation function for reasons
        # of symmetry, monotonicity, and so forth. I have not checked this as
        # of yet.
        # ***
        return F.gelu(out + self.skip(x))

class ManifoldRanker(nn.Module):
    """
    The manifold ranker architecture.

    This is a Multi-Task Learning model. It does not just output a rank;
    it learns a model of the data through reconstruction.

    Structure:
        1. Input Fusion: Combines raw data with Positional Encodings.
        2. Encoder: A stack of ResGCNLayers to capture local topology.
        3. Head A (Decoder): Reconstructs input (Unsupervised / Regularization).
```

```
        4. Head B (Ranker): Predicts the utility score (Supervised).
    """
    def __init__(self, feature_dim):
        super().__init__()

        # Input Projection
        # We concatenate features + positional encodings to give the model
        # both "Context" (what the data is) and "Location" (where it is in the graph).
        self.input_proj = nn.Linear(feature_dim + Config.POS_ENC_DIM, Config.HIDDEN_DIM)

        # Graph Encoder
        self.layers = nn.ModuleList([
            ResGCNLayer(Config.HIDDEN_DIM, Config.HIDDEN_DIM)
            for _ in range(Config.NUM_LAYERS)
        ])

        # Normalization layer stabilizes training dynamics (prevents exploding gradients)
        self.norm = nn.LayerNorm(Config.HIDDEN_DIM)

        # Reconstruction Head (The "Autoencoder")
        # Used for LAMBDA_RECON loss. Ensures the latent space 'h' actually
        # retains the information of the input cities.

        # ***
        # ATTENTION:
        # I am not sure if the ELU activation function follows all the properties
        # need. This needs to be checked!
        # ***
        self.decoder = nn.Sequential(
            nn.Linear(Config.HIDDEN_DIM, Config.HIDDEN_DIM),
            nn.ELU(),
            nn.Linear(Config.HIDDEN_DIM, feature_dim)
        )

        # Ranking Head (scorer)
        # Used for LAMBDA_RANK and LAMBDA_COMPASS.
        # Compresses the latent representation into a bottleneck (Latent Dim)
        # and then down to a single scalar score in the range [0, 1].

        # ***
        # ATTENTION:
        # The tanh activation function is well-behaved theoretically, but it
        # is dangerous, because it often causes gradient vanishing. Hence,
        # maybe we will have to eventually change this activation function
        # for something that is better-performing.
        # ***
        self.rank_mlp = nn.Sequential(
            nn.Linear(Config.HIDDEN_DIM, Config.LATENT_DIM),
            nn.Tanh(), # Tanh centers activations [-1, 1], good for manifold coordinates
            nn.Linear(Config.LATENT_DIM, 1)
        )

    def forward(self, x, pe, edge_index):
        """
```

```
        Forward pass of the network.

        ***
        IMPORTANT:
        Transitivity is guaranteed here because we map to a single REAL scalar.
        ***

        Args:
            x: Input features
            pe: Positional Encodings (Spectral/PCA coordinates)
            edge_index: Graph connectivity
        """
        # Fuse Features + Position
        h = torch.cat([x, pe], dim=1)
        h = F.gelu(self.input_proj(h))

        # Pass through Graph Layers
        for layer in self.layers:
            h = layer(h, edge_index)

        # Final embedding representation
        h = self.norm(h)

        # Calculate outputs
        scores = torch.sigmoid(self.rank_mlp(h)) # Sigmoid forces output to [0, 1] range
        recon = self.decoder(h)

        return scores, recon

    def enforce_monotonicity(self):
        """
        Domain Knowledge Injection: Monotonicity Constraint.

        In ranking problems, we often know that more is better (e.g., more 5G coverage
        is strictly better than less).

        However, Neural Networks are universal function approximators and might
        accidentally learn and infer some other logic that infringes this basic
        rule due to noise.

        Hence, this function clamps the weights of the final ranking layer to be POSITIVE.
        This forces the derivative to be positive, meaning that increasing the latent
            quality
        must increase the final score.

        ***
        ATTENTION:
        At some point this 'enforce_monotonicity' function will need to be improved,
        beccause clamping weights reduces generalization, restricting the Neural Network
        and diminishing its full potential. I believe that if we could guarantee that our
        neural network is the integral of a function that has the features of the system
        as inputs, then our Neural Network could work as a sort of CDF, and would always
        be monotonically increasing with respect to the features, providing generality
        and guaranteeing monotonicity in a more elegant way.
```

```
        ***

        """
        with torch.no_grad():
            self.rank_mlp[-1].weight.data.clamp_(min=0.0)


# ==============================================================================
# EXECUTION ORCHESTRATOR
# ==============================================================================
def main():
    """
    Main Execution Loop.

    Architecture:
    1. Load and process data into System RAM.
    2. Pinned Memory: Lock specific RAM blocks for fast GPU transfer.
    3. Training:
        - Sample a random batch of 'Center Nodes'.
        - Expand a small subgraph around them (on CPU).
        - Stream ONLY that subgraph to GPU.
        - Calculate Loss -> Backpropagate -> Update.
    4. Inference:
        - Chunk the dataset.
        - Predict scores piece-by-piece.
        - Stitch results back together.
    """
    start_global = time.time()

    # Reproducibility
    Config.set_seeds()
    print(f"[Setup] Device: {Config.DEVICE} | VRAM Safe Mode")

    # Data Loading
    # The dataset class handles the normalization (Log1p, RobustScaler).
    dataset = MunicipalityManifold(Config.CSV_FILE_PATH)
    X_model, X_pareto, ids = dataset.load()
    if X_model is None: return

    # MEMORY OPTIMIZATION: PINNED MEMORY
    # Python stores data in virtual memory (paged).
    # To move it to GPU, CPU must first copy it to a "pinned" (locked) buffer, then to GPU.
    # Optimization: We create the tensors directly in Pinned Memory.
    # Result:
    # 1. Faster transfer speeds.
    # 2. Allows asynchronous transfer (CPU prepares next batch while GPU crunches current one).
    # 3. We DO NOT move .to(device) yet. The big matrix stays in RAM.
    X_tens_cpu = torch.from_numpy(X_model).pin_memory()
    X_par_tens_cpu = torch.from_numpy(X_pareto).pin_memory()

    # Topology Setup
    # Calculate the KNN graph structure on the CPU.
```

```
    engine = SafeGraphEngine(X_model, k=Config.KNN_K)

    # Positional encodings also stay in Pinned RAM.
    pe_tens_cpu = torch.from_numpy(engine.pos_enc).pin_memory()

    # Model Setup
    # Only the Model weights stay permanently on the GPU VRAM.
    model = ManifoldRanker(feature_dim=X_model.shape[1]).to(Config.DEVICE)
    optimizer = optim.AdamW(model.parameters(), lr=Config.LR, weight_decay=Config.
        WEIGHT_DECAY)

    # MarginRankingLoss:
    # Creates a criterion that measures the loss given inputs 'x1', 'x2',
    # which are two 1D-tensors, and also given a label tensor 'y'
    # (containing 1 or -1).
    # If y = 1, we want 'x1' > 'x2' by at least a quantity (a 'margin').
    margin_loss_fn = nn.MarginRankingLoss(margin=Config.MARGIN)

    print(f"\n[Optimization] Training ({Config.ITERATIONS} steps)")
    model.train()
    start_time = time.time()

    # TRAINING LOOP
    for step in range(1, Config.ITERATIONS + 1):

        # Stochastic Sampling (the focus of local analysis)
        # We pick random cities to focus on.
        center_nodes = np.random.randint(0, X_model.shape[0], Config.BATCH_SIZE)

        # Subgraph Expansion (CPU Work)
        # We find the neighbors of these randomly chosen cities.
        # This function handles the "Max Neighbor" limit to prevent VRAM explosion.
        # Returns:
        # sub_nodes_idx: Global IDs of all nodes in this small graph.
        # sub_edge: Connectivity matrix for this small graph.
        sub_nodes_idx, sub_edge, local_idx = engine.sample_safe_subgraph(center_nodes,
            Config.NUM_LAYERS)

        # Data Streaming
        # We perform a "Gather" operation on the CPU Pinned Tensor.
        # .to(..., non_blocking=True) allows the CPU to proceed immediately without
            waiting for transfer.
        sub_x = X_tens_cpu.index_select(0, sub_nodes_idx).to(Config.DEVICE, non_blocking=
            True)
        sub_pe = pe_tens_cpu.index_select(0, sub_nodes_idx).to(Config.DEVICE, non_blocking
            =True)
        sub_par = X_par_tens_cpu.index_select(0, sub_nodes_idx).to(Config.DEVICE,
            non_blocking=True)
        sub_edge = sub_edge.to(Config.DEVICE, non_blocking=True)
        local_idx = local_idx.to(Config.DEVICE, non_blocking=True)

        # Forward Pass
        # The model only sees the small subgraph.
        scores, recon = model(sub_x, sub_pe, sub_edge)
```

```
        # Loss Calculation
        # We only care about the scores of the "Center Nodes" we selected.
        # The neighbors exist only to provide context (message passing).
        s_batch = scores[local_idx].flatten()
        naive_proxy = sub_par[local_idx].mean(dim=1)

        # Loss 1: Compass Loss (Soft Monotonicity)
        # Roughly points in the direction of the raw statistics.
        compass_loss = F.mse_loss(s_batch, naive_proxy)

        # Loss 2: Reconstruction Loss (Regularization)
        # To remember what the original data looked like.
        recon_loss = F.mse_loss(recon[local_idx], sub_x[local_idx])

        # Loss 3: Rank Loss (Pairwise Optimization)
        # If City A has better stats than City B, Rank(A) must be > Rank(B).
        # We create random pairs within the batch.
        perm = torch.randperm(s_batch.size(0))
        s_A, s_B = s_batch, s_batch[perm] # Model Scores
        u_A, u_B = naive_proxy, naive_proxy[perm] # Ground Truth Proxies

        # Target is 1 if A > B, -1 if B > A
        target = torch.sign(u_A - u_B)

        # Masking: avoid learning from pairs that are too similar (noise).
        # Only learn if the proxy difference is significant (e.g., > 0.05).
        mask = (torch.abs(u_A - u_B) > 0.05).float()
        rank_loss = (margin_loss_fn(s_A, s_B, target) * mask).mean()

        # Loss 4: Geometric Loss (Manifold Smoothness)
        # We want neighbors to have similar scores.
        if sub_edge.size(1) > 0:
            src, dst = sub_edge
            geo_loss = torch.mean((scores[src] - scores[dst])**2)
        else:
            geo_loss = torch.tensor(0.0, device=Config.DEVICE)

        # Weighted Sum of Objectives
        total = (Config.LAMBDA_RANK * rank_loss) + \
                (Config.LAMBDA_RECON * recon_loss) + \
                (Config.LAMBDA_GEO * geo_loss) + \
                (Config.LAMBDA_COMPASS * compass_loss)

        # Backward Pass
        total.backward()

        # Gradient Accumulation (Virtual Batch Size)
        # We update weights only every 4 steps.
        # This simulates a batch size of BATCH_SIZE * GRAD_ACCUM_STEPS,
        # while using (1/GRAD_ACCUM_STEPS)-th the VRAM.
        if step % Config.GRAD_ACCUM_STEPS == 0:
            # Gradient Clipping: Prevents "exploding gradients" in GNNs

```

```
        # ***
        # ATTENTION:
        # I believe there might be a better way to do this!
        # ***
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)

        optimizer.step()

        # Constraint Injection: Force rank weights to be positive
        model.enforce_monotonicity()

        # 'set_to_none=True' is slightly faster than 'zero_grad()'
        # as it avoids writing zeros to memory.
        optimizer.zero_grad(set_to_none=True)

        # Logging and VRAM Maintenance
        if step % 200 == 0:
            elapsed = time.time() - start_time
            # Monitor VRAM to ensure safety measures are working
            vram = torch.cuda.memory_allocated()/1e6 if torch.cuda.is_available() else 0
            print(f"Step {step:04d} | Loss: {total.item():.4f} | Nodes: {len(sub_nodes_idx)} | VRAM: {vram:.0f}MB")
            start_time = time.time()
            # Explicit cache clearing to prevent fragmentation
            torch.cuda.empty_cache()

    # INFERENCE LOOP
    print("\n[Inference] Global Ranking")
    model.eval()
    final_scores = np.zeros(X_model.shape[0], dtype=np.float32)
    chunk_size = 512 # Micro-batching for inference

    with torch.no_grad():
        for i in range(0, X_model.shape[0], chunk_size):
            chunk = np.arange(i, min(i+chunk_size, X_model.shape[0]))

            # Even for inference, we must construct the graph context.
            # We use 1 layer for speed, assuming local context is enough for final scoring.
            sub_nodes, sub_edge, local_idx = engine.sample_safe_subgraph(chunk, num_layers=1)

            sub_x = X_tens_cpu.index_select(0, sub_nodes).to(Config.DEVICE, non_blocking=True)
            sub_pe = pe_tens_cpu.index_select(0, sub_nodes).to(Config.DEVICE, non_blocking=True)
            sub_edge = sub_edge.to(Config.DEVICE, non_blocking=True)

            s, _ = model(sub_x, sub_pe, sub_edge)

            # Map local subgraph scores back to global array
            final_scores[chunk] = s[local_idx].cpu().numpy().flatten()

```

```
            if i % (chunk_size*10) == 0:
                print(f"Processed {i} / {X_model.shape[0]}...")
                # Aggressive cleanup prevents memory creep during long loops
                del sub_x, sub_pe, sub_edge
                torch.cuda.empty_cache()

    # Saving Results
    # Reload original CSV to attach results
    df_out = pd.read_csv(Config.CSV_FILE_PATH, sep=';', encoding='utf-8-sig', decimal='.'
        )

    # Ensure lengths match if rows were dropped during loading
    if len(df_out) != len(final_scores):
        df_out = df_out.iloc[:len(final_scores)]

    df_out['Utility_Score'] = final_scores
    df_out['Rank'] = df_out['Utility_Score'].rank(ascending=False).astype(int)
    df_out = df_out.sort_values('Rank')

    # Validation: Spearman Correlation
    # Check how much the learned rank correlates with the naive Pareto proxy.
    # High correlation (>0.8) means the model respected the data.
    # Low correlation might indicate the model found hidden non-linear patterns
    # (or failed).
    proxy = np.mean(X_pareto, axis=1)
    corr, _ = spearmanr(final_scores, proxy)
    print(f"Spearman Correlation: {corr:.4f}")

    df_out.to_csv("Final_Rankings.csv", index=False)

    print(f"Finished in {time.time()-start_global:.1f}s")


main()
```